

jepctl: Putting Joint-Embedding Predictive Architectures in Practitioners’ Hands

An engineering report on a local runtime for JEPA encoders

Younss

Independent developer

`younssiga@gmail.com`

September 21, 2026

Abstract

This is an engineering report, not a research contribution. Joint-Embedding Predictive Architectures (JEPAs) are an active research programme [9, 1, 2, 10, 3], and the checkpoints and results they produce are published openly. Turning them into something a practitioner can actually run — against their own camera, on their own laptop, wired into their own application — is a separate, unglamorous amount of work that nobody had done. `jepctl` is that work: a single-binary local runtime, written in Rust with no Python dependency, that pulls published JEPA-style checkpoints, reports exactly how much of each one loaded, and serves embeddings over a REST and SSE API, a command line, and a desktop testbench. On top of it I packaged four worked demonstrations, each built only from operations the research already justifies: few-shot visual and audio recognition with a per-patch divergence map; an online latent world model over the live camera that predicts the next embedding and surfaces its prediction error as a surprise signal, following the loop described in LeWorldModel [10]; a robot arm driven by posture mirroring or by minimising a latent distance to a goal image; and a multimodal companion that fuses vision and audio locally. I trained nothing, invented no method, and ran no comparative evaluation. What I report instead is what an integrator needs to know: the checkpoint coverage obtained for six published encoders, the latencies and memory footprints measured on one machine, one memory-retention defect that cost a factor of seven in resident memory until it was found, and an explicit account of what this class of model does not do. `jepctl` is Apache-2.0 licensed and available at <https://github.com/younss/jepctl>.

1 Introduction

A predictive model of the world is useful only if something can run it. The JEPA line of work [9, 1, 2] argues that a system should learn by predicting *representations* of what it has not yet seen rather than reconstructing pixels; recent work trains such a model stably end to end from raw pixels [10] and argues the principle carries far beyond vision [3]. The science is being done, and done in the open: the papers are public, the checkpoints are downloadable.

What is missing is the last mile. A practitioner who wants to point I-JEPA or V-JEPA 2 at their own webcam assembles a Python environment, writes their own preprocessing, guesses at the checkpoint’s tensor naming, and discovers only at inference time whether any of it was right. Language models crossed that gap when single-binary runtimes turned “pull a model, run it, call an API” into one line [11]. Representation learning has had no equivalent.

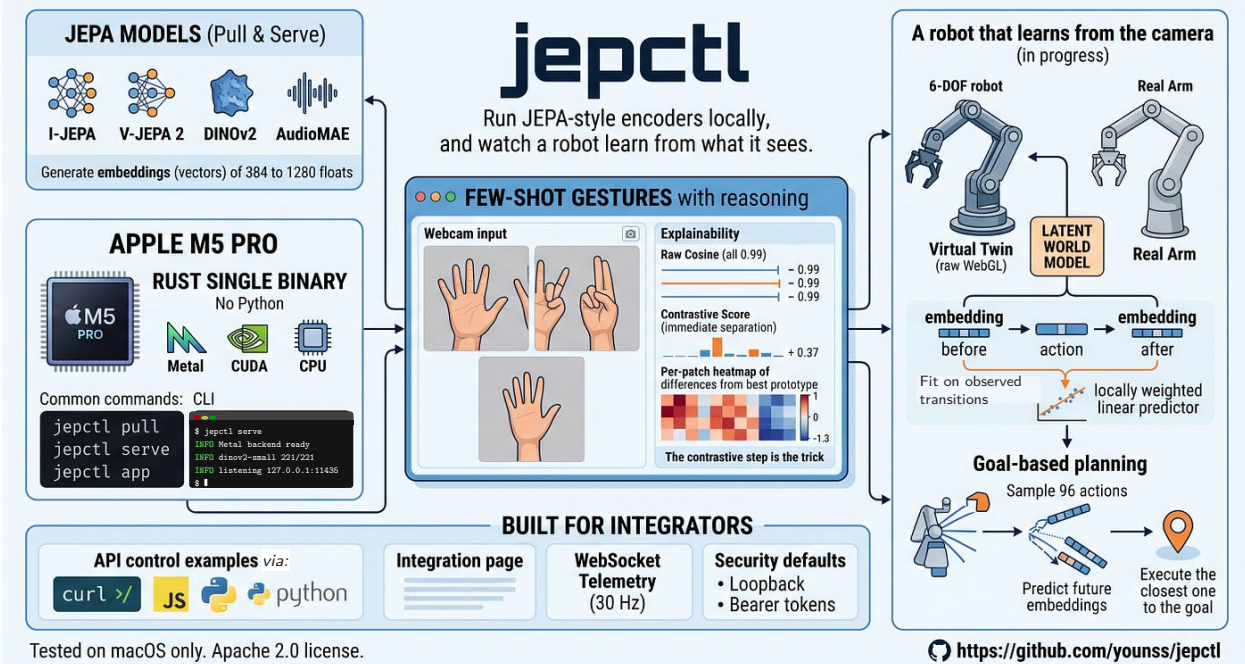


Figure 1: Overview of `jepctl`. Left: catalogued JEPA-style encoders (I-JEPA, V-JEPA 2, DINOv2, AudioMAE) producing embeddings of 384 to 1280 dimensions, served by a single Rust binary with Metal, CUDA, or CPU backends and a small command line. Centre: few-shot recognition with its reasoning exposed, including the raw cosine scores, the contrastive score that separates otherwise near-identical classes, and the per-patch difference map. Right: the camera-driven robot loop, in which embeddings before and after an action train a locally weighted linear predictor that is then used to plan by sampling candidate actions and executing the one predicted to land closest to the goal. Bottom: the integration surface and the security defaults. Figure by the author.

`jepctl` is an attempt to build one. It is a single Rust binary with no Python dependency that downloads and loads JEPA-style checkpoints with an explicit report of how many tensors were covered, serves embeddings over HTTP, a live SSE stream, and a command line, and ships a desktop testbench with four worked demonstrations.

1.1 Scope: what this paper is, and what it is not

I want to be precise about the genre, because it determines how the rest of this document should be read.

This paper is an engineering report. Its subject is a tool: its design, the decisions behind it, the defects found while building it, and measurements taken on one machine. It is offered to practitioners and integrators who want to use JEPA encoders in an application, and to researchers who would like their published checkpoints to be easier for other people to run.

It is not a research contribution. I did not train a model, propose a method, prove anything, or run a comparative evaluation against baselines. Every representation used here comes from a checkpoint someone else published and trained. Where this report describes a loop or a formula, it is describing *what the software computes and which published work it is taken from*, not a new result. In particular:

- The predict-then-measure-surprise loop of Section 5 is the one described by Maes et al. [10]; my implementation deliberately weakens it (frozen encoder, small online predictor) so that it runs on a laptop and can be watched in seconds. That substitution makes it a demonstration, not a reproduction, and certainly not an improvement.
- The goal-reaching controller of Section 6 minimises a latent distance and plans by rolling out a fitted transition model, which is the structure used in [10]. The fitting is deliberately simple; no claim is made about control performance.
- The design constraint that nothing above the encoder should assume a modality is taken from the argument in [3]. I do not implement their method.

The contribution, such as it is, is availability. Published research becomes useful to a business when someone can run it in an afternoon, see what it does, and call it from their own code. The numbers in Section 8 are engineering figures — latency, resident memory, checkpoint coverage — reported so an integrator can plan, not so a method can be ranked.

2 What this tool builds on

This section records which published results `jepctl` depends on and what it takes from each. It is not a survey, and it does not attempt to explain the theory better than the original papers do; a reader who wants the science should read them.

The encoders I package. `jepctl` runs checkpoints, and does not train them. The catalogued encoders are I-JEPA [1], V-JEPA 2 [2], DINOv2 [12], plain ViT [4], and AudioMAE [5]. The practical property all of them provide, and that every demonstration in this report relies on, is a patch-level representation in which semantically similar content lands close together and nuisance variation matters less than it does in pixel space. That property is theirs, not mine.

LeWorldModel [10]: where the World loop comes from. Maes et al. train an encoder and a predictor end to end from pixels with two loss terms: a mean-squared next-embedding prediction loss, and SIGReg, a regulariser that projects embeddings onto random directions and applies a normality test to each one-dimensional projection so the latent distribution is pushed toward an isotropic Gaussian, which is what keeps the training from collapsing. Two things in that paper are the reason Section 5 exists: the predictor is autoregressive in latent space, and the resulting prediction error is a usable *surprise* signal that flags implausible events. I reuse the shape of that loop. I do not reuse the training: my encoder is frozen and my predictor is a small online fit, which means none of their stability contribution is being tested or reproduced here.

JEPA-Anything [3]: a design constraint. Cui et al. argue the JEPA principle is domain-agnostic and validate a factorised predictive design across seven domains. I implement none of it. What I took is an architectural rule for the runtime: nothing above the encoder should assume a modality. In `jepctl` an image, a video clip, and an audio spectrogram are the same object — an input that becomes an embedding — and the matcher, the world model, and the control loops are written against embeddings rather than pixels.

Tooling precedent. The shape of the product is borrowed from local language-model runtimes [11]: one binary, a model catalogue, pull and serve, an HTTP API. Weights are read via `safetensors` [6]; inference uses `candle` [7].

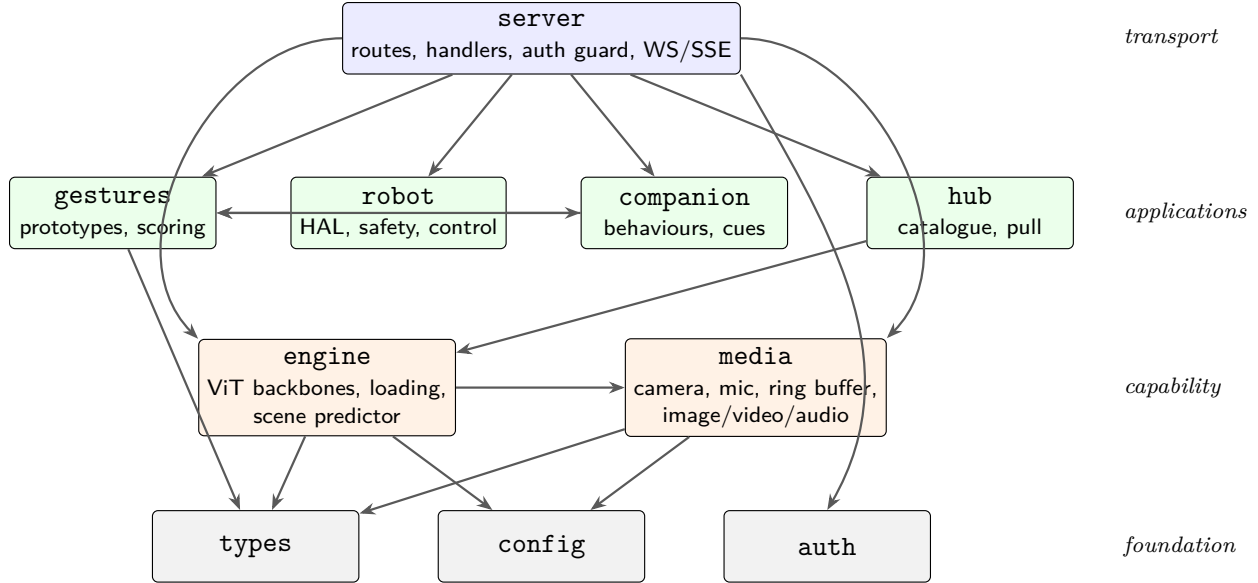


Figure 2: Module dependencies. Arrows point from a module to what it uses. Application modules depend on **engine** and **media** but never on **server**, so the same perception and control code is reachable from the CLI, the HTTP API, and the tests without a running daemon.

What this class of model does not do. JEPA encoders are not generative: they do not decode a latent back to pixels. They do not estimate metric depth. These are not limitations of my implementation but properties of the models, and both the software’s interface and Section 9 say so explicitly, because a visualisation that implies otherwise would mislead a user who is deciding whether the approach fits their problem.

3 Architecture

`jepctl` is written in Rust and builds to one binary of roughly 14000 lines across eleven top-level modules. Inference uses **candle** [7] with Metal, CUDA, or multi-threaded CPU backends selected at runtime. The HTTP layer is **axum**; the desktop testbench is a native window hosting a dependency-free web UI. This section documents the structure, the runtime behaviour, and the interactions between components, since those are the parts an integrator has to reason about. Figure 1 gives the overall picture; Figure 4 the dataflow.

3.1 Module structure

Figure 2 shows the modules and the direction of their dependencies. The layering is deliberate: perception and control modules know nothing about HTTP, and the transport layer knows nothing about tensors. Table 1 states each module’s responsibility.

One departure from strict layering is worth naming: **types** references application modules because several API data types embed domain structures (a robot telemetry snapshot, a gesture score). It is a convenience that keeps the wire format in one file, at the cost of a dependency edge pointing the wrong way.

Module	Responsibility	Lines
engine	ViT backbones (plain, CLS, DINOv2, V-JEPA 2 tubelets, audio), checkpoint mapping and coverage, the active-model registry, the online scene predictor	2 400
media	Camera and microphone supervisors, frame ring buffer, region of interest, image/video/audio decoding and preprocessing	1 700
server	Routes, handlers, authentication guard, SSE stream, WebSocket telemetry, embedded UI assets	3 600
robot	Backend trait and two implementations, safety supervisor, control loop, latent world model, simulated camera view	1 800
companion	Behaviour state machine, cue memory, pose blending, 30 Hz loop	750
gestures	Prototype store, contrastive scoring, bundles	620
hub	Model catalogue, manifests, download	720
auth, config, types	Tokens and roles, paths and settings, shared data types	1 020

Table 1: Modules and their responsibilities. Line counts are rounded and include tests that live beside the code.

3.2 Runtime view: processes, threads and tasks

`jepctl` is one operating-system process. Inside it, Figure 3 shows the concurrent activities: two OS threads owning hardware that cannot be moved across threads, and six asynchronous tasks on the Tokio runtime.

All shared state is reached through a single `AppState` handle cloned into every request: the engine, the model catalogue, the authentication manager, the camera and microphone supervisors, the frame ring buffer, the gesture and sound stores, the region of interest, the robot handle, the companion handle, and the scene predictor. Each is an `Arc` over the appropriate lock: `RwLock` where readers dominate (ring buffer, prototype stores), `Mutex` where every access mutates (robot core, scene predictor), and `watch` channels to broadcast telemetry without contention.

3.3 Interaction: pulling and loading a model

Figure 5 traces the path that decides whether the runtime will serve a model at all. The coverage check is the only place in the system that can refuse work on correctness grounds, and it does so before a single embedding is produced.

3.4 Interaction: the live detection stream

Figure 6 shows the loop that drives the few-shot demonstrations. The property worth noting for an integrator is that registration and matching traverse the same path: a prototype is captured through exactly the pipeline that will later be used to score it, so a crop, a region of interest, or a model change cannot silently desynchronise the two.

3.5 Interaction: the closed control loop

Figure 7 is the only place where the system closes a loop through the physical world: the controller acts, waits for the arm to settle, observes the consequence through the camera, and learns from the

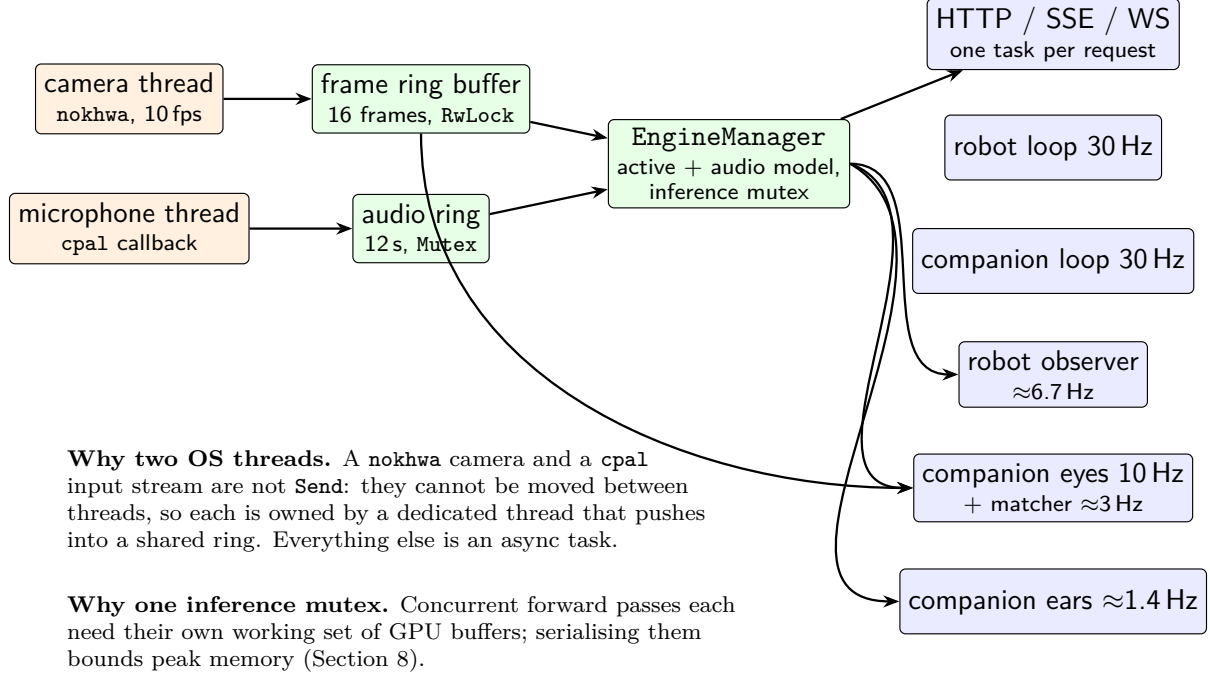


Figure 3: Runtime view. Orange: OS threads owning non-movable hardware handles. Blue: Tokio tasks. Green: shared state. Every consumer reaches the model through one `EngineManager` whose inference mutex serialises forward passes.

difference. It is also where the safety supervisor sits between intent and actuation.

3.6 Strict checkpoint loading

A partially loaded Vision Transformer [4] is a silent failure: the forward pass succeeds, the output has the advertised dimension, and every downstream metric is meaningless. `jepctl` therefore maps every parameter of the constructed backbone against the checkpoint, trying the naming conventions of Hugging Face ViT/I-JEPA, Hugging Face DINOv2, Hugging Face V-JEPA 2, and timm-style fused QKV, and reshaping tensors whose element count matches (for instance a `Conv3d` patch kernel feeding a linear tubelet embedding). It then returns a coverage report. A model whose coverage is below 100% is refused with HTTP 422 rather than served. Positional embeddings are adapted separately: the CLS row is sliced or synthesised as required, and the patch grid is resampled bicubically when the checkpoint resolution differs from the configured one.

Weights are read from `safetensors` [6], which is parsed and memory-mapped but never executed; this removes the arbitrary-code-execution surface of pickle-based checkpoints. Table 2 lists the coverage obtained for the catalogued encoders.

3.7 Modality-agnostic input path

Images are centre-cropped and resized to the model input with the normalisation declared by the manifest (ImageNet or Inception statistics). Video clips are decoded natively for GIF and animated WebP and through an external `ffmpeg` binary for MP4/WebM, then uniformly sub-sampled to the clip length the model expects; image models embed each sampled frame and average. Audio is down-mixed to mono, resampled to 16kHz, and turned into a Kaldi-style log-mel filterbank of

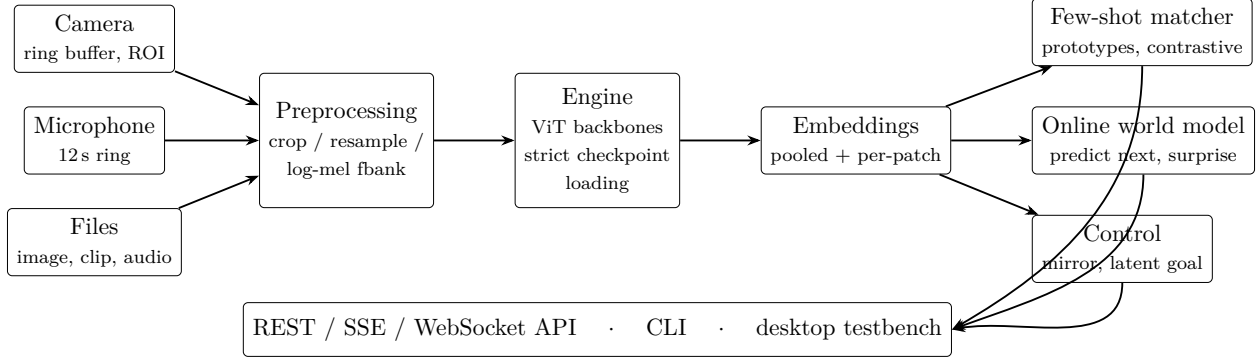


Figure 4: `jepctl` architecture. Every module above the encoder operates on embeddings only; none of them reads pixels. The same path serves the command line, the HTTP API, and the desktop testbench.

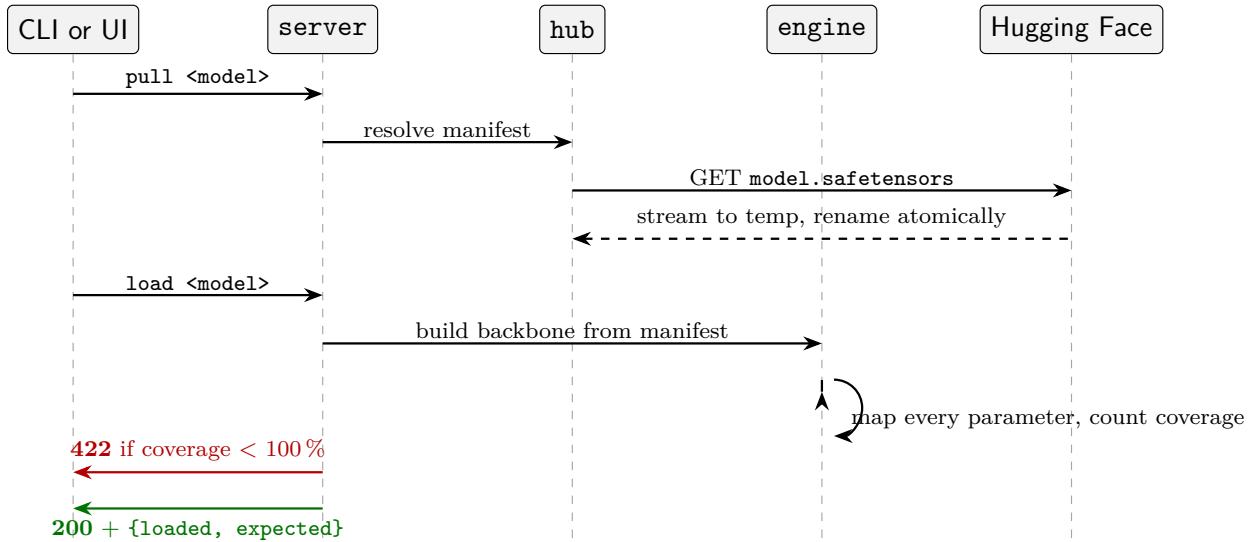


Figure 5: Pulling and loading. The runtime never serves a partially loaded model: coverage below 100 % is an error carrying the count and the first missing tensor names, not a warning in a log.

1024 × 128 for the AudioMAE front end [5]. Because that encoder’s CLS token was never trained as a summary, the manifest selects mean pooling; using CLS yields near-identical vectors for every input, which is exactly the kind of silent failure the coverage report is designed to prevent.

3.8 Deployment posture

The daemon binds 127.0.0.1 by default and can be switched to all interfaces from a settings toggle, in which case authentication is mandatory. Bearer tokens are stored as SHA-256 digests and compared in constant time; no CORS headers are emitted by default, so a web page open in the user’s browser cannot read camera frames or embeddings. Decoders are bounded (8192 px per side, 256 MB of decoder allocation, 512 frames or 256 million pixels per animation) so that a decompression bomb is refused before it is allocated.

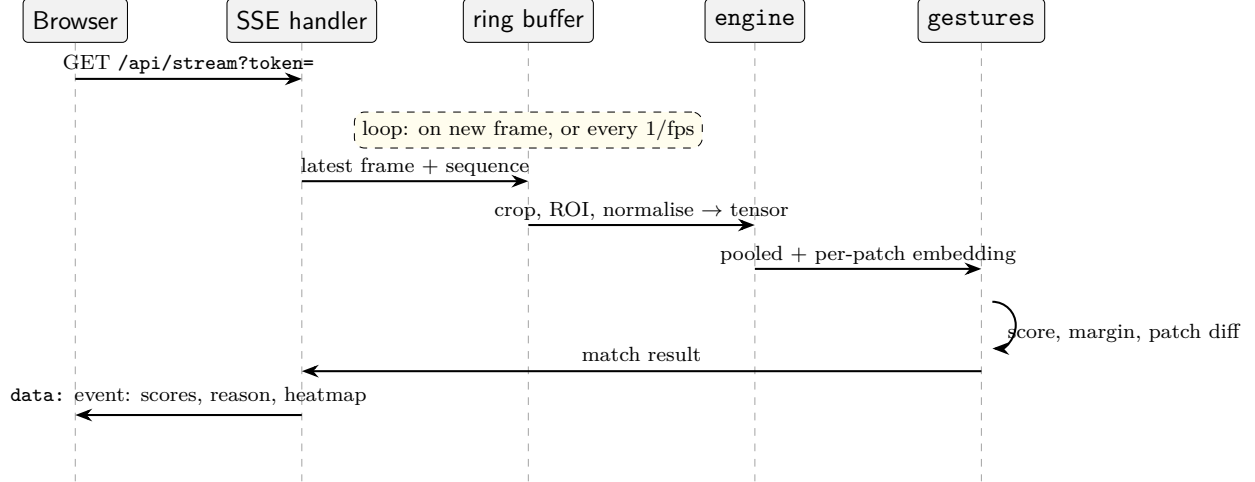


Figure 6: The detection stream. Frames that have not changed are skipped by sequence number, so an idle scene costs no inference. The same handler feeds detections to the robot when it is in a gesture-driven mode.

Checkpoint	Family	Dim	Coverage
facebook/ijepa_vith14_1k	I-JEPA ViT-H/14	1280	516/516
facebook/dinov2-small	DINOv2 ViT-S/14	384	221/221
facebook/vjepa2-vitl-fpc64-256	V-JEPA 2 ViT-L	1024	388/388
google/vit-base-patch16-224	ViT-B/16	768	197/197
timm ViT-B/16 (fused QKV)	ViT-B/16	768	197/197
gaunernst/vit_base_patch16_1024_128.audiomae_as2m	AudioMAE ViT-B	768	197/197

Table 2: Checkpoint coverage reported by `jepctl` for the catalogued encoders. Coverage is the number of backbone parameters populated from the checkpoint over the number the architecture declares; anything below 100 % is refused at load time.

4 Demonstration 1: few-shot inspection and recognition

The first application registers a small number of reference embeddings per class and classifies by similarity. Let $\{z_i^{(c)}\}$ be the ℓ_2 -normalised embeddings captured for class c and p_c their normalised mean. For a query z , a naive matcher scores $\cos(z, p_c)$. That score is dominated by whatever the classes share — the room, the lighting, the operator’s torso. `jepctl` therefore also computes a contrastive score in the centroid-removed subspace: with $\bar{p} = \frac{1}{C} \sum_c p_c$, it scores $\cos(z - \bar{p}, p_c - \bar{p})$, and reports a blend $s_c = 0.35 \cos(z, p_c) + 0.65 \cos(z - \bar{p}, p_c - \bar{p})$, gated by a threshold and a minimum lead over the runner-up. Removing the shared component before comparing prototypes is standard practice in few-shot classification rather than anything devised here; what the tool adds is that the blend, the threshold, and the margin are exposed in the interface and in the exported bundle, so an integrator can see and reproduce the decision. A dedicated *neutral* class absorbs frames in which nothing intentional is present, which is what prevents an empty scene from being forced into the nearest class.

Localising the deviation. Because the encoder emits one vector per patch, the same machinery localises *where* a query departs from its prototype: the per-patch distance to the best-ranked

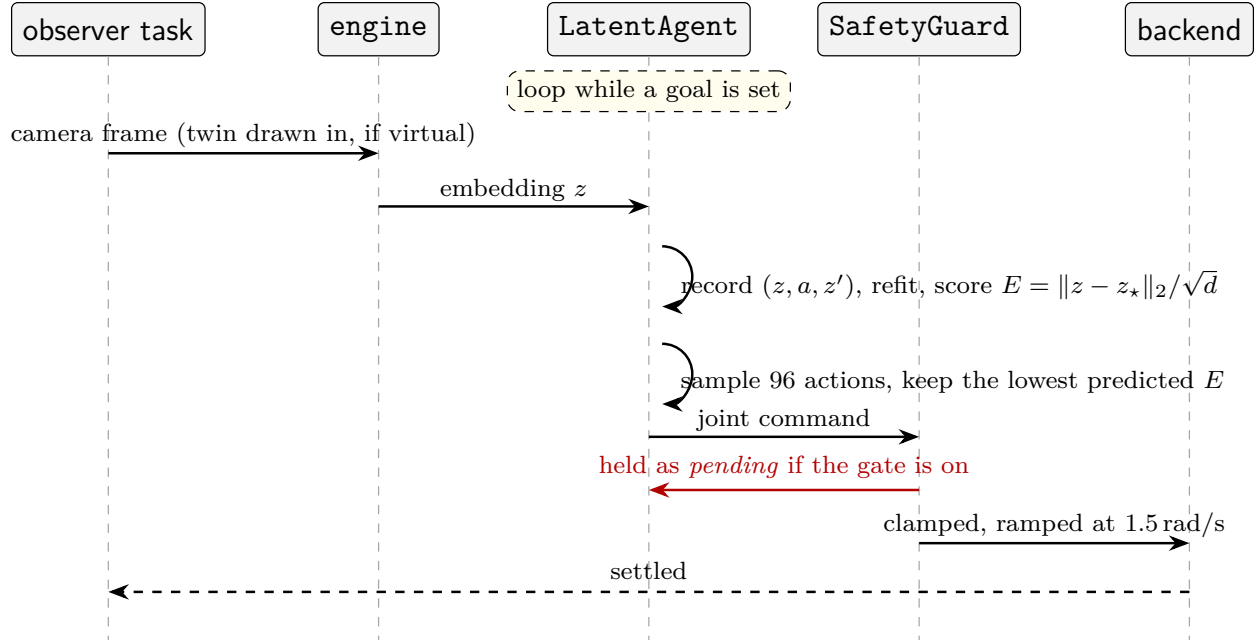


Figure 7: The goal-seeking loop. The agent never commands the backend directly: every command passes the safety supervisor, which clamps it to the joint limits, ramps it, and on the physical backend parks it until a human approves.

class’s patch prototype, arranged on the 16×16 ViT grid, is a divergence map. In the industrial-inspection scenario shipped with the tool — a bottle, capped or not — two short captures are enough to separate the two states, and the divergence map concentrates on the neck. Nothing was ever trained to look for a cap; the localisation is a consequence of patch-level semantics. This is the property that makes the approach economically interesting: the classical alternative requires a labelled dataset and a training run per defect class.

Why this is worth a practitioner’s attention. Glass and plastic produce specular highlights that move with the operator and the ambient light. Pixel differencing responds strongly to those high-frequency luminance changes; the patch embeddings are comparatively stable under them, because the pretraining objective rewards representations that survive nuisance variation. This is reported as an observed behaviour of the shipped scenario on one desk, under one set of lights. It is not a robustness result: establishing that would take a dataset, a protocol, and a baseline, none of which are part of this work.

5 Demonstration 2: an online latent world model over the camera

The World module implements the LeWorldModel loop [10] with one deliberate substitution: the encoder is frozen (any catalogued JEPA encoder) and only the predictor is fit, online, on the live stream. That substitution throws away the representation-learning part of [10] — precisely the part their SIGReg term exists to protect — in exchange for something a user can watch behave in seconds on a laptop, with no training run and no checkpoint to distribute. It is a demonstration of the idea, not an implementation of their method. Figure 8 shows the loop.

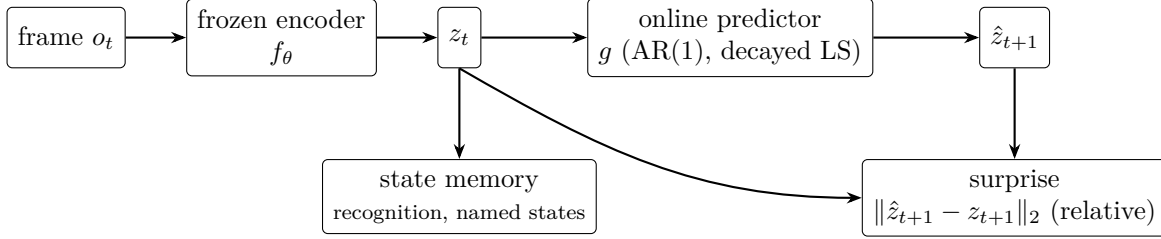


Figure 8: The World loop. The encoder is frozen; the predictor is fit online from the stream. The prediction error of the previous step, normalised by the recent typical error, is the surprise signal used for anomaly detection, following [10].

Predictor. The pooled embedding is projected onto a fixed random basis of dimension $K = 48$; by the Johnson–Lindenstrauss lemma [8] this preserves distances well enough for the error statistic while keeping the online fit cheap and stable. In that space a per-dimension first-order autoregression $x_{t+1} = a \odot x_t + c$ is fit by decayed least squares (decay 0.97), falling back to persistence until enough transitions have accumulated.

Surprise. Let $e_t = \|\hat{z}_t - z_t\|_2$ be the error of the prediction made at $t - 1$, and $\bar{e}$ an exponentially weighted estimate of the recent typical error. The reported surprise is $\text{clip}((e_t/\bar{e} - 1)/1.5, 0, 1)$, so that a scene the model tracks well sits near zero and a genuine deviation spikes. This is the operationalisation of the surprise evaluation in [10]: prediction error in latent space, not in pixel space, is what flags the implausible event. Per patch, the same comparison against a constant-velocity prediction $2z_t - z_{t-1}$ yields a spatial surprise map, which the interface renders as a glow on the diverging region and which drives a configurable alarm with a timestamped, spatially localised incident log.

Recognition and named states. A bounded memory of normalised pooled embeddings gives a recognition score (nearest neighbour by cosine), and novel states are appended, so the system reports whether the current view is familiar. Users may additionally save named reference states (“empty desk”, “full shelf”), after which the module reports which one it sees or flags the view as unknown.

Rendering it without overclaiming. The reconstruction shown to the user maps the live frame onto a surface whose relief is derived from the semantic distance of each patch to a background prototype (the mean of the patches whose distance to the frame centroid is below the median). A smoothstep transfer keeps the background flat and extrudes only the foreground, and the patch grid is upsampled bicubically so silhouettes are smooth. The interface labels this as what it is: the real frame given shape by what the encoder perceives. It is neither a generated image nor a metric depth map.

6 Demonstration 3: an arm driven from latent space

The robot module drives a 6-DoF arm with a gripper through a hardware abstraction trait with two implementations: an in-memory backend feeding a WebGL twin, and a serial backend speaking a Feetech STS3215 register protocol. A safety supervisor enforces per-joint limits, a 1.5 rad/s velocity ramp at 30 Hz, and an emergency stop that only an administrator can reset; with the physical

backend every command is held until explicitly approved. Two control modes are latent-space operations.

Mirror: teleoperation without skeleton tracking. The operator demonstrates a handful of postures, each associated with an arm pose. At run time the arm does not snap to the winning class; it moves to the softmax-weighted blend of the demonstrated poses, with weights $w_c \propto \exp((s_c - \max_j s_j)/\tau)$, $\tau = 0.06$, candidates more than 0.20 below the best receiving zero weight and the blend suppressed entirely below a score of 0.35. Raising an arm slowly therefore sweeps the joint targets continuously through the demonstrated poses. Unlike pose-estimation pipelines, no joint of the human body is ever localised: the comparison is between whole-posture embeddings.

Visual goal reaching: energy minimisation in latent space. Following the planning structure of [10], the arm captures a goal embedding $z_\star$ from the camera and then minimises

$$E(z, z_\star) = \frac{\|z - z_\star\|_2}{\sqrt{d}}, \quad (1)$$

where d is the embedding dimension. A latent transition model is fit online from exploration: triples (z, a, z') collected while the arm babbles are used to fit a locally weighted ridge regression $\Delta z = W[a; 1]$, refit after every observation. At each step the controller samples 96 candidate actions, predicts their outcome *inside the learned model*, executes the one with the lowest predicted energy plus exploration noise, observes the real result, and learns from it. Until enough transitions exist the policy is random, and the telemetry reports which regime is active together with predicted versus observed energy, so the user can judge the model rather than trust it. Convergence is declared against a noise floor measured from two observations of a static scene, and a plateau detector reports when a single viewpoint cannot disambiguate further.

Because the virtual arm is invisible to a real camera, the observation used in simulation is the camera frame with the twin’s kinematics rendered into it over a frozen background, so that the only thing changing between observations is the arm itself. This is a simulation convenience, and the interface says so; with the physical backend the observation is the raw camera frame.

7 Demonstration 4: local multimodal interaction

The companion is a virtual character that fuses two frozen encoders locally. Motion attention is computed from the centroid of the thresholded difference between consecutive 32×24 grayscale thumbnails at 10 Hz and steers the head; visual cues are matched at roughly 3 Hz against the gesture prototypes; audio cues are matched at roughly 1.4 Hz from the last 1.5 s of microphone audio, zero-padded into the AudioMAE window. Recognised cues fire behaviours with a two-second cooldown, and postures mapped to body poses are blended by the same softmax rule as the arm. A 30 Hz control loop renders the result. The point of this module is the absence of the usual machinery: there is no transcription model, no keyword spotter, and no object detector — a clap and a wave are both nearest-prototype lookups over local embeddings.

8 Engineering measurements

The numbers below are the ones an integrator asks for before committing to a technology: does it fit in memory, how fast does a frame come back, and did the weights actually load. They were

Quantity	Setting	Observed
World endpoint latency	DINOv2-S, full frame, relief and texture	12–46 ms per call
Gesture stream latency	DINOv2-S, model view	12–15 ms per frame
V-JEPA 2 clip forward	ViT-L, 16 frames at 256 px, peak RSS	1.98 GB
Daemon under load	V-JEPA 2 loaded, SSE stream and 4 concurrent captures	1.5 GB peak
Goal reaching (simulated arm)	after ≈ 45 s of exploration	all 6 joints within ≈ 0.1 rad in 15–30 s
Latent transition model fit	simulated arm, frozen background	fit error ≈ 0.05

Table 3: Single-machine measurements.

taken on a single machine (Apple M5 Pro, 15 logical cores, 24 GB unified memory, macOS, Metal backend) during interactive use of the shipped application.

They are *not* an evaluation. There is no baseline, no dataset, no protocol, and no statistical treatment; a second run on a different machine would produce different figures. Nothing here should be read as a claim about the models themselves, whose accuracy was established by the papers that trained them.

A memory bug worth reporting. Every model was originally constructed from a parameter map whose tensors were autograd-tracked variables, because that is the natural way to copy a checkpoint into a pre-built architecture. Inference through such tensors records a backward graph, and that graph keeps every intermediate activation of every layer alive until the output is dropped. Instrumenting the Metal allocator showed GPU allocation climbing linearly by roughly 0.5 GB per transformer layer, reaching 13.5 GB after 24 layers for a single V-JEPA 2 clip; with concurrent requests the process reached 36 GB and was killed. Rebuilding each model on detached tensors after loading — storage is shared, so nothing is copied — reduced the peak resident set for one clip forward to 1.98 GB, of which 1.2 GB is the weights themselves. It is recorded here because the failure mode is invisible in a framework that assumes training: nothing is wrong except that inference is paying for a gradient nobody will ever use.

What was checked, and what was not. The repository contains 76 Rust unit and integration tests, including in-memory HTTP tests of every API surface, and 4 UI tests. What has *not* been done is a numerical comparison against the reference PyTorch implementations, because the project has no Python dependency. Implementations follow the reference code structure, including V-JEPA 2’s tiled sine/cosine rotary tables with interleaved rotation and the Kaldi Povey window for the audio front end, and determinism and discriminativeness are checked, but a bit-level equivalence claim would be unsupported, so none is made.

9 Limitations

- **Nothing here is validated as science.** The demonstrations show that the operations run and behave sensibly on one desk; they do not establish accuracy, robustness, or generality. Anyone deciding whether this family of models suits their problem should read the papers that trained them and run their own trial on their own data.
- **Not generative, no metric depth.** JEPA encoders do not decode latents to pixels and do not estimate distance. The 3D reconstruction in the World module is the real camera image

given relief by semantic foreground separation; calling it depth would be wrong.

- **The online predictor is weak by construction.** A per-dimension AR(1) fit in a 48-dimensional random projection is adequate to expose the predict-then-measure-surprise loop and to detect deviations, but it is not the learned predictor of [10] and makes no claim to their control performance.
- **Single viewpoint.** Goal reaching can plateau at a pose that is view-equivalent to the goal; the system reports the plateau rather than pretending to converge.
- **Few-shot means few-shot.** Prototypes interpolate between what was demonstrated and do not extrapolate; recognition degrades under large changes of viewpoint, scale, or illumination relative to the captures.
- **No published Audio-JEPA checkpoint.** The audio path is complete, but the verified audio encoder is AudioMAE [5]; a JEPA-trained audio encoder in a loadable format would be a manifest entry away.
- **The serial robot backend is frame-tested only.** The STS3215 protocol is implemented from the register map and unit-tested at the frame level; it has not been run against physical hardware by the authors.

10 Conclusion

The JEPA literature has moved quickly, from a training principle [9] to stable end-to-end world models [10] and a claim of domain generality [3]. That work is not mine and this report does not extend it.

What I found missing was smaller and more mundane: between a published checkpoint and a practitioner with a laptop and a camera sits a day of environment wrangling, an undocumented tensor-naming convention, and no way to tell whether the model that just loaded is the one the authors trained. `jepctl` removes that day. Pull a checkpoint, be told exactly how much of it loaded, and then reach for recognition, anomaly detection, or control as operations on embeddings, from a command line or an HTTP call.

The demonstrations it ships are deliberately ordinary — inspect a bottle, mirror a posture, reach a pictured goal, react to a clap — and none of them is novel. That is the point. Each one costs a handful of captures rather than a labelled dataset, runs entirely on the machine in front of you, and can therefore be shown to someone who has to decide whether this family of models is worth their time. Making research legible and runnable is not a scientific result, but it is the step at which published work starts being used, and somebody has to do it.

Availability

Source code, documentation, and the model catalogue are available under the Apache-2.0 licence at <https://github.com/younss/jepctl>. The measurements in Section 8 correspond to version 0.3.0.

Acknowledgements

The author thanks the maintainers of the open-source checkpoints catalogued by this tool.

References

- [1] Mahmoud Assran, Quentin Duval, Ishan Misra, Piotr Bojanowski, Pascal Vincent, Michael Rabbat, Yann LeCun, and Nicolas Ballas. Self-supervised learning from images with a joint-embedding predictive architecture. *arXiv preprint arXiv:2301.08243*, 2023.
- [2] Mido Assran, Adrien Bardes, David Fan, Quentin Garrido, Russell Howes, Mojtaba Komeili, Matthew Muckley, Ammar Rizvi, Claire Roberts, Koustuv Sinha, Artem Zholus, Sergio Arnaud, Abha Gejji, Ada Martin, Francois Robert Hogan, Daniel Dugas, Piotr Bojanowski, Vasil Khalidov, Patrick Labatut, Francisco Massa, Marc Szafraniec, Kapil Krishnakumar, Yong Li, Xiaodong Ma, Sarath Chandar, Franziska Meier, Yann LeCun, Michael Rabbat, and Nicolas Ballas. V-JEPA 2: Self-supervised video models enable understanding, prediction and planning. *arXiv preprint arXiv:2506.09985*, 2025.
- [3] Taoyong Cui, Zhongyao Wang, Xinyue Xu, Weiyang Liu, Zhaochen Yu, Yuying Zhang, Qiang Gao, Mengyue Yang, Wanli Ouyang, Pheng Ann Heng, Yingcheng Wu, Zhenfei Yin, and Ling Yang. JEPA-Anything: Learning predictive models across different worlds. *arXiv preprint arXiv:2609.20800*, 2026.
- [4] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale. In *International Conference on Learning Representations (ICLR)*, 2021.
- [5] Po-Yao Huang, Hu Xu, Juncheng Li, Alexei Baevski, Michael Auli, Wojciech Galuba, Florian Metze, and Christoph Feichtenhofer. Masked autoencoders that listen. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
- [6] Hugging Face. Safetensors: a simple, safe format for storing tensors. <https://github.com/huggingface/safetensors>, 2023.
- [7] Hugging Face. Candle: a minimalist ml framework for rust. <https://github.com/huggingface/candle>, 2024.
- [8] William B. Johnson and Joram Lindenstrauss. Extensions of Lipschitz mappings into a Hilbert space. *Contemporary Mathematics*, 26:189–206, 1984.
- [9] Yann LeCun. A path towards autonomous machine intelligence. Technical report, OpenReview, 2022. Version 0.9.2.
- [10] Lucas Maes, Quentin Le Lidec, Damien Scieur, Yann LeCun, and Randall Balestriero. LeWorld-Model: Stable end-to-end joint-embedding predictive architecture from pixels. *arXiv preprint arXiv:2603.19312*, 2026.
- [11] Ollama. Ollama: get up and running with large language models locally. <https://github.com/ollama/ollama>, 2024.
- [12] Maxime Oquab, Timothée Darcet, Théo Moutakanni, Huy V. Vo, Marc Szafraniec, Vasil Khalidov, Pierre Fernandez, Daniel Haziza, Francisco Massa, Alaaeldin El-Nouby, Mahmoud Assran, Nicolas Ballas, Wojciech Galuba, Russell Howes, Po-Yao Huang, Shang-Wen Li, Ishan Misra, Michael Rabbat, Vasu Sharma, Gabriel Synnaeve, Hu Xu, Hervé Jegou, Julien Mairal, Patrick Labatut, Armand Joulin, and Piotr Bojanowski. DINOv2: Learning robust visual features without supervision. *arXiv preprint arXiv:2304.07193*, 2023.